

Lab Manual: Creating Job-Triggered ETL Endpoints with Flask and PySpark

Objective

By the end of this lab, you will:

- Set up a Flask backend with two endpoints:
 - `/run-etl` → triggers a PySpark ETL job.
 - `/get-status` → tracks the job's progress.
- Integrate **PySpark for data processing** and **Flask for orchestration**.
- Use **threading** to run ETL jobs asynchronously.

Prerequisites

- Python 3.7+ installed.
- Install Flask and PySpark:

```
pip install flask pyspark
```

- A code editor (VS Code, PyCharm, etc.).

Step 1: Set Up Project Folder

```
mkdir etl_flask_spark  
cd etl_flask_spark
```

Inside the folder, create this structure:

```
etl_flask_spark/  
|— app.py  
|— etl_job.py  
|— data.csv
```

Step 2: Create a Sample Dataset

Create a file named **data.csv**:

```
id,quantity,price  
1,2,10  
2,5,7  
3,3,20  
4,1,15
```

This is the input dataset that PySpark will process.

Step 3: Write the PySpark ETL Job

Create a file **etl_job.py** and paste this code:

```
import time  
from pyspark.sql import SparkSession  
from pyspark.sql.functions import col  
  
# Global job status tracker  
status = {"running": False, "message": "ETL job not started."}  
  
def run_etl():  
    status["running"] = True  
    status["message"] = "ETL job started."  
  
    try:
```

```

# Create Spark session
spark = SparkSession.builder \
    .appName("FlaskETLJob") \
    .master("local[*]") \
    .getOrCreate()

time.sleep(1) # Simulate delay
df = spark.read.csv("data.csv", header=True, inferSchema=True)

time.sleep(1) # Transformation stage
df_transformed = df.withColumn("total", col("quantity") *
col("price"))

time.sleep(1) # Write output
df_transformed.write.mode("overwrite").csv("output_data",
header=True)

status["message"] = "ETL job completed successfully."
spark.stop()

except Exception as e:
    status["message"] = f"ETL job failed: {str(e)}"
finally:
    status["running"] = False

def get_status():
    return status

```

Explanation

- `status` → dictionary that tracks whether the job is running and its state.
- `run_etl()` →
 - Starts Spark session.
 - Reads `data.csv`.
 - Adds a new column `total = quantity * price`.
 - Writes output to `output_data/`.
- `get_status()` → returns the current job status.

Step 4: Write the Flask App

Create `app.py`:

```
from flask import Flask, jsonify
import threading
from etl_job import run_etl, get_status

app = Flask(__name__)

# Endpoint to trigger ETL job
@app.route("/run-etl", methods=["POST"])
def trigger_etl():
    if get_status()["running"]:
        return jsonify({"message": "ETL job already running"}), 400

    # Run ETL job in a background thread
    thread = threading.Thread(target=run_etl)
    thread.start()

    return jsonify({"message": "ETL job triggered."})

# Endpoint to check ETL status
@app.route("/get-status", methods=["GET"])
def check_status():
    return jsonify(get_status())

if __name__ == "__main__":
    app.run(debug=True)
```

Explanation

- `/run-etl` →
 - Starts the ETL job in a **background thread**.
 - Prevents blocking Flask while Spark runs.
- `/get-status` →

- Returns the current job status as JSON.
- `app.run(debug=True)` → runs the Flask app locally.

Step 5: Run the Flask App

Start the Flask server:

```
python app.py
```

Output:

* Running on <http://127.0.0.1:5000>

Step 6: Test the Endpoints

1. Trigger ETL job:

```
curl -X POST http://127.0.0.1:5000/run-etl
```

Response:

```
{"message": "ETL job triggered."}
```

2. Check job status:

```
curl http://127.0.0.1:5000/get-status
```

Sample responses while job runs:

```
{"running": true, "message": "ETL job started."}
```

After completion:

```
{"running": false, "message": "ETL job completed successfully."}
```

3. Check output folder:

```
ls output_data
```

You should see CSV files with the transformed `total` column.

Verification Checklist

- Dataset `data.csv` created.
- `etl_job.py` implemented with PySpark ETL logic.
- `app.py` exposes `/run-etl` and `/get-status`.
- Job triggers asynchronously.
- Status endpoint reports progress.
- Output CSV files generated in `output_data/`.

Summary

In this lab, you:

- Built a **PySpark ETL pipeline**.
- Wrapped it inside a **Flask backend**.
- Exposed endpoints `/run-etl` and `/get-status`.
- Ran jobs asynchronously using **threading**.

This is a **mini orchestration system** for data pipelines, a foundation for job schedulers, monitoring dashboards, or API-driven data workflows.